

ANÁLISIS Y ESTUDIO DEL PROYECTO ENV-DB-ARCHITECTURE

Programa: Ingeniería De Sistemas

Asignatura: Arquitectura De Software

Profesor: Jesús Ariel Gonzales

Integrantes

María Sofía Aljure Herrera

Brayan Smith Bedoya

Corporación Universitaria Del Huila Corhuila

Neiva – Huila

Noviembre 2025

Contenido	
Introducción	3
¿Qué es el repositorio y para qué sirve?	3
¿Cómo está organizada la carpeta principal?	3
Dentro de cada proyecto se encuentra esta estructura	4
Qué significa cada una de estas cosas encontradas en la estructura:	4
environments/	4
Un ejemplo de config/.env que es el global podría ser:	5
Cómo se usa	5
Descripción de los ambientes	5
Docker Compose (archivo docker-compose.yml)	6
Explicación de cada sección.....	6
Scripts principales	7
El repositorio incluye varios scripts en PowerShell (.ps1) que automatizan tareas de configuración, despliegue y auditoría. A continuación, se describen los más importantes:	7
init-project.ps1	7
sync-config.ps1	7
Sincroniza el archivo config/.env con los demás .env (dev, qa, staging, prod).	7
manage.ps1	7
Sirve para manejar los contenedores Docker fácilmente.	7
validate-config.ps1	8
audit-changelog.ps1 y audit-alert.ps1	8
Backups	8
Cómo agregar un nuevo proyecto	8
Integración CI/CD	9
Principios de Diseño Aplicados	10
Tecnologías Centrales	10
Conclusiones	13

Introducción

Este repositorio es una plantilla y laboratorio para gestionar bases de datos PostgreSQL en varios proyectos y varios ambientes (desarrollo, QA, staging y producción).
No es solo código: es una forma ordenada de administrar bases de datos con automatizaciones, respaldos, seguridad y scripts que evitan errores manuales.

En otras palabras: es un paso a paso y una caja de herramientas para montar, configurar, validar, auditar y respaldar bases de datos PostgreSQL por proyecto y por ambiente.
Sirve para que varios proyectos (por ejemplo, inventory, penalty, security) compartan la misma forma de trabajar sin mezclar datos sensibles ni repetir configuración.

¿Qué es el repositorio y para qué sirve?

Pensemos que este repositorio es una caja de herramientas que permite crear y manejar bases de datos PostgreSQL sin enredarse con comandos manuales.

Sirve para tener:

- Un entorno de base de datos de desarrollo (dev)
- Otro de pruebas (qa)
- Otro de staging (preproducción)
- Y el de producción (prod)

Todo eso, pero para varios proyectos distintos como:

- inventory → base de datos del inventario
- penalty → base de datos de sanciones o penalizaciones
- security → base de datos de usuarios, roles o autenticación

¿Cómo está organizada la carpeta principal?

La estructura principal del repositorio está organizada en tres secciones principales, cada una con una función específica:

Carpeta	Qué tiene	Para qué sirve
architecture/	Diagramas que muestran cómo se conectan los contenedores, redes, y backups.	Sirve para entender visualmente la estructura técnica.
projects/	Donde están los proyectos reales (inventory, penalty, security).	Aquí es donde se crean y ejecutan las bases de datos.

Archivos raíz (README.md, ARCHITECTURE.md, ADD_PROJECT.md, etc.)	Documentos que explican cómo usar todo el sistema y scripts que verifican o auditan.	Ayudan a configurar y mantener el sistema ordenado y seguro.
---	--	--

Dentro de cada proyecto se encuentra esta estructura

En este caso esta es la estructura de inventory, que se repite igual en penalty y security:

```

projects/inventory/
├── environments/
│   ├── config/.env
│   ├── dev/.env
│   ├── dev/docker-compose.yml
│   ├── qa/.env
│   ├── qa/docker-compose.yml
│   ├── staging/.env
│   ├── staging/docker-compose.yml
│   ├── prod/.env
│   └── prod/docker-compose.yml
├── backups/
│   ├── dev/
│   ├── qa/
│   ├── staging/
│   └── prod/
├── Dockerfile
├── README.md
├── init-project.ps1
├── manage.ps1
├── sync-config.ps1
└── .gitignore

```

Qué significa cada una de estas cosas encontradas en la estructura:

environments/

el environments guarda la configuración del proyecto.

Ahí están los .env que son las variables de entorno y los docker-compose.yml que definen los contenedores.

Hay dos tipos de archivos .env:

Archivo	Qué contiene	Ejemplo
config/.env	Que es la configuración general que se	PROJECT_NAME=shopping-cart

	aplica a todos los ambientes.	
dev/.env, qa/.env, etc.	Que son las variables específicas para ese ambiente (como contraseña o base de datos).	POSTGRES_PASSWORD=dev_Str0ng_S3cr3t_2024

Un ejemplo de config/.env que es el global podría ser:

```
PROJECT_NAME=shopping-cart
DB_NAME_BASE=shopping_cart
POSTGRES_USER=postgres
PORT_PROD=5432
PORT_STAGING=5433
PORT_QA=5434
PORT_DEV=5435
```

Cómo se usa

1. Editas el archivo config/.env
2. Luego ejecutas el comando:

```
.\sync-config.ps1
```

Y este comando copia esa configuración global a los .env de dev, qa, staging y prod. Así no se tiene que editar uno por uno.

3. Luego se puede levantar la base de datos con:

```
.\manage.ps1 dev up
```

(que iniciaría el contenedor Docker del ambiente dev.)

Descripción de los ambientes

Ambiente	Uso	Puerto	Base de datos
Dev	Desarrollo diario (se puede borrar y recrear)	5435	shopping_cart_dev
QA	Pruebas de calidad	5434	shopping_cart_qa
Staging	Pruebas antes de producción	5433	shopping_cart_staging
Prod	Producción real	5432	shopping_cart_prod

Docker Compose (archivo docker-compose.yml)

El archivo docker-compose.yml actúa como una receta de construcción. Define los servicios, redes y volúmenes necesarios para levantar una instancia de PostgreSQL en cada ambiente de forma independiente.

Cada ambiente del proyecto (desarrollo, pruebas, staging y producción) cuenta con un archivo denominado **docker-compose.yml**, el cual define la forma en que se debe construir y ejecutar el contenedor de **PostgreSQL** correspondiente a dicho entorno.

Este archivo actúa como una plantilla de configuración, permite levantar una instancia de PostgreSQL completamente configurada para cada ambiente, con sus propias variables, red y almacenamiento, asegurando la separación y consistencia entre entornos de desarrollo, prueba y producción.

Ejemplo de un archivo docker-compose.yml:

```
services:
  postgres-dev:
    container_name: shopping-cart-postgres-dev
    build: .
    env_file:
      - ../config/.env
      - ./dev/.env
    ports:
      - "5435:5432"
    volumes:
      - shopping_cart_dev_data:/var/lib/postgresql/data
```

Explicación de cada sección

- **services:**
Indica la lista de servicios (contenedores) que se crearán. En este caso, solo se define uno, llamado postgres-dev, que representa la base de datos del entorno de desarrollo.
- **container_name:**
Es el nombre asignado al contenedor de Docker. Facilita la identificación del servicio al listar los contenedores activos.
Ejemplo: shopping-cart-postgres-dev.
- **build:**
Especifica que Docker debe utilizar el archivo Dockerfile presente en el mismo directorio (.) para construir la imagen personalizada del contenedor. Esta imagen incluye las configuraciones necesarias de PostgreSQL, como idioma, versión y parámetros locales.
- **env_file:**
Define los archivos que contienen las variables de entorno requeridas para el

servicio.

En este caso:

- `../config/.env` → variables globales del proyecto.
- `../dev/.env` → variables específicas del entorno de desarrollo (como contraseña y nombre de base de datos).
- **ports:**
Asigna los puertos para la comunicación entre el sistema anfitrión y el contenedor. En el ejemplo, el puerto **5435** del equipo local se enlaza con el puerto interno **5432** del contenedor (puerto estándar de PostgreSQL).
- **volumes:**
Define un volumen persistente donde se almacenan los datos de la base de datos, garantizando que la información no se pierda al detener o eliminar el contenedor. En este caso, el volumen `shopping_cart_dev_data` se vincula con la ruta interna `/var/lib/postgresql/data`.

Scripts principales

El repositorio incluye varios scripts en PowerShell (.ps1) que automatizan tareas de configuración, despliegue y auditoría. A continuación, se describen los más importantes:

init-project.ps1

Sirve para inicializar o renombrar un proyecto.

Ejemplo:

```
.\init-project.ps1 -ProjectName "nuevo"
```

→ Cambia el nombre del proyecto en todas partes, sincroniza todo y levanta los contenedores.

sync-config.ps1

Sincroniza el archivo `config/.env` con los demás `.env` (dev, qa, staging, prod).

Así si cambias el puerto o el nombre del proyecto en el archivo global, se actualiza en todos.

manage.ps1

Sirve para manejar los contenedores Docker fácilmente.

Comando	Qué hace
<code>.\manage.ps1 dev up</code>	Levanta el ambiente dev
<code>.\manage.ps1 dev down</code>	Apaga el contenedor dev
<code>.\manage.ps1 all backup</code>	Hace respaldo de todas las bases de datos

<code>.\manage.ps1 qa rebuild</code>	Reconstruye el ambiente de QA
--------------------------------------	-------------------------------

`validate-config.ps1`

Revisa si todo está bien configurado:

- Que existan los `.env` y `docker-compose.yml`
- Que tengan usuario, contraseña y base de datos
- Que las contraseñas no sean débiles

`audit-changelog.ps1` y `audit-alert.ps1`

Sirven para auditar cambios:

- `audit-changelog.ps1` guarda un registro (con hashes) del estado de los archivos importantes.
- `audit-alert.ps1` compara el estado actual con el anterior para ver si alguien cambió algo sospechoso.

Que es hashes: es como una huella digital de un archivo. Permite saber si un archivo fue modificado, porque si cambia algo, su huella (el hash) también cambia.
Se usa para comprobar que los archivos estén intactos y no hayan sido alterados.

Backups

Cada proyecto tiene su propia carpeta `backups/`, dividida por ambiente. Allí se almacenan los respaldos de las bases de datos (`.sql`) generados en cada entorno.

Importante:

- Nunca se deben subir backups ni contraseñas a Git.
- Se recomienda cifrar los backups con `openssl`.

Ejemplo:

```
openssl aes-256-cbc -in backup.sql -out backup.sql.enc -k LA_CLAVE
```

Que es openssl: una herramienta de línea de comandos utilizada para cifrar y proteger archivos sensibles, como copias de seguridad de bases de datos.
Su uso garantiza la confidencialidad de los datos almacenados o compartidos.

Cómo agregar un nuevo proyecto

Para agregar un nuevo proyecto solo se necesita seguir algunos pasos y usar algunos comandos

1. Copia la carpeta example/ y se renombra (por ejemplo, sales).
2. Cambiar el PROJECT_NAME en config/.env.
3. Ejecutar:

```
.\init-project.ps1 -ProjectName "sales"
```

4. Validar con:

```
.\validate-config.ps1
```

5. Levantar el ambiente dev:

```
.\manage.ps1 dev up
```

Integración CI/CD

El archivo CI_CD_EXAMPLE.md contiene un ejemplo práctico de cómo automatizar el despliegue de un ambiente PostgreSQL utilizando GitHub Actions. Este flujo permite validar, construir y desplegar contenedores sin intervención manual.

En este flujo:

1. **Descarga el repositorio** desde GitHub.
2. **Prepara Docker** para construir los contenedores.
3. **Levanta el ambiente de PostgreSQL** usando el archivo docker-compose.yml.
4. **Ejecuta el script validate-config.ps1** para comprobar que la configuración sea correcta.

Este proceso puede adaptarse a cualquier proyecto o ambiente (por ejemplo, dev, qa o prod).

- **CI (Integración Continua):** proceso que verifica automáticamente que los cambios del código no generen errores al unirse con el proyecto.
- **CD (Despliegue Continuo):** proceso que publica automáticamente esos cambios en los entornos de prueba o producción después de ser validados.

En conjunto, CI/CD garantiza que los cambios en el proyecto sean probados y desplegados de forma rápida, segura y repetible.

Que es GitHub Actions: es una herramienta de automatización que ofrece GitHub.

Permite crear flujos de trabajo que se ejecutan automáticamente cuando ocurre algo en el repositorio. Se usa para compilar, probar y desplegar proyectos sin hacerlo manualmente, ideal para implementar procesos de **CI/CD**.

Principios de Diseño Aplicados

Patrón/Principio	Herramientas en el Proyecto	Beneficio Logrado
Infraestructura como Código (IaC)	docker-compose.yml, Dockerfile, manage.ps1	La infraestructura es reproducible y gestionada mediante código, no manualmente.
Aislamiento de Entornos	Configuración de puertos y variables de entorno distintas por cada ambiente (dev, qa, staging, prod).	Se evita la mezcla de datos y la afectación de un entorno a otro, lo cual es crítico en la separación de datos de producción.
Arquitectura de Microservicios	Carpeta projects/ con estructuras independientes para inventory, penalty, y security.	Permite la descentralización y la evolución independiente de la base de datos de cada servicio.
Auditoría y Trazabilidad	audit-changelog.ps1, audit-alert.ps1	Mantiene un registro inmutable (hash) de los cambios, crucial para la seguridad y el cumplimiento normativo.

Tecnologías Centrales

Tecnología	Rol en la Arquitectura
PostgreSQL	Es el motor de base de datos elegido, conocido por su robustez, cumplimiento con estándares SQL y características avanzadas de replicación y extensibilidad.
Docker Compose	Actúa como el orquestador local que define y gestiona el ciclo de vida de los contenedores (construcción, inicio, detención y eliminación).
PowerShell (.ps1)	Es la capa de abstracción de comandos. Simplifica y automatiza tareas complejas de Docker y gestión de archivos (.env), facilitando el trabajo del equipo de <i>DevOps</i> .

Microservicio backend-security

Este conjunto de archivos muestra cómo se construye y despliega un microservicio Java usando Spring Boot y Maven en un contenedor, y cómo ese proceso se automatiza con Jenkins (CI/CD).

Dockerfile

El Dockerfile define las instrucciones necesarias para construir la imagen del contenedor donde se ejecutará el microservicio.

En este caso, establece el entorno base con Java y Maven, copia el código del proyecto, configura las dependencias y compila la aplicación.

Su función principal es convertir el código fuente en una aplicación lista para ejecutarse dentro de un contenedor, asegurando que siempre se ejecute bajo las mismas condiciones sin importar el sistema operativo o equipo donde se despliegue.

Línea	Explicación	¿Por Qué es Importante?
<code>FROM ubuntu:20.04</code>	Base: Define que el contenedor comenzará con la imagen base de Ubuntu 20.04.	Proporciona un sistema operativo ligero y estable para ejecutar la aplicación.
<code>ENV TZ="America/Bogota"</code>	Zona Horaria: Configura la zona horaria a la de Bogotá.	Es crucial para el registro de eventos y las operaciones de base de datos.
<code>RUN apt install openjdk-17-jre-headless</code>	Java: Instala el Java Runtime Environment (JRE) versión 17. El <i>headless</i> significa que no instala componentes gráficos, lo cual es ideal para un servidor <i>backend</i> .	Es el motor que permite ejecutar la aplicación Spring Boot.
<code>ARG MAVEN_VERSION=3.8.8...</code>	Maven: Descarga, instala y configura la herramienta de construcción Maven.	Maven es el encargado de leer el código Java, gestionar las librerías (<i>dependencies</i>) y

		compilarlo en el archivo final <code>.jar</code> .
<code>COPY ./devops/docs/...</code>	Configuración de Maven: Copia scripts especiales (<code>mvn-entriypoint.sh</code>) y el archivo de configuración de Maven (<code>settings-docker.xml</code>).	Permite adaptar Maven para que funcione perfectamente dentro del entorno de Docker (ej. dónde guardar el caché de librerías).
<code>WORKDIR /app-compiled</code>	Directorio de Trabajo: Establece la carpeta <code>/app-compiled</code> como el lugar donde se harán las operaciones.	Es la carpeta raíz del proyecto dentro del contenedor.
<code>COPY . .</code>	Código Fuente: Copia todo el código fuente de tu proyecto local al contenedor.	Traslada el código que necesita ser compilado.
<code>RUN cp -r ... application.properties</code>	¡Paso Crítico de Configuración! Copia el archivo <code>properties</code> de tu carpeta <code>devops</code> y lo renombra como <code>application.properties</code> en la carpeta de recursos de Spring Boot.	Fija la configuración (conexión a DB, puerto, secretos) que la aplicación usará al momento de compilar.
<code>RUN mvn clean install -Dmaven.test.skip=true</code>	Compilación (CI): Ejecuta Maven. El comando compila el código y genera el archivo <code>.jar</code> ejecutable. Se salta las pruebas unitarias (<code>-Dmaven.test.skip=true</code>) para un <i>build</i> más rápido, aunque esto es cuestionable en un buen CI.	Es el paso de Integración Continua donde se crea el producto final de <i>software</i> .
<code>CMD java -jar ...</code>	Ejecución: Indica el comando final que se ejecutará al iniciar el contenedor. Llama a Java para ejecutar el <code>.jar</code> compilado.	Es la instrucción para arrancar el microservicio.

EXPOSE 9000	Puerto Interno: Informa que la aplicación Java escucha internamente en el puerto 9000.	Documentación para que Docker sepa qué puerto puede mapear.
-------------	--	---

devops/docker-compose.yml:

El archivo docker-compose.yml tiene como propósito orquestar y coordinar la ejecución de los distintos servicios que conforman el sistema.

Define los contenedores que deben iniciarse, los puertos que utiliza cada uno, las redes de comunicación entre ellos y las dependencias del entorno.

En este caso, permite que el servicio de backend-security funcione junto a una base de datos PostgreSQL dentro de una misma red, especificando el puerto (9001) para su acceso externo.

En resumen, este archivo facilita la ejecución y conexión de todos los componentes del sistema de forma estructurada y automatizada.

Línea	Explicación Detallada	Conexión con el Proyecto
image: \${CONTAINER_IMAGE}	Imagen a Usar: Toma el nombre y la etiqueta (backend-security:251104...) que generó el Jenkinsfile.	Versionamiento: Garantiza que se despliegue la versión más reciente y etiquetada del código.
ports: - "9001:9000"	Mapeo de Puertos: Mapea el puerto interno 9000 (de la aplicación Java) al puerto externo 9001 del servidor.	Es la forma en que los usuarios o tu Frontend se conectarán al <i>backend</i> de seguridad.
networks: - network_local_server	Red: Conecta este contenedor a la red network_local_server.	¡Paso Crucial! Esta debe ser la misma red donde está corriendo tu contenedor de PostgreSQL. Así, el <i>backend</i> puede <i>hablar</i> con la base de datos.

devops/properties/properties:

El archivo `properties` contiene las configuraciones esenciales para el funcionamiento del microservicio.

Incluye parámetros como el puerto del servidor, las credenciales y el nombre de la base de datos, la clave para la autenticación JWT, y otras propiedades de seguridad o conexión.

Su función es proporcionar la información necesaria para que la aplicación se comuniquen correctamente con los recursos externos, adaptándose fácilmente a diferentes entornos (desarrollo, pruebas o producción).

Propiedad	Valor en el Archivo	Función
<code>server.port= 9000</code>	Puerto de la Aplicación: Confirma el puerto en el que Spring Boot va a escuchar internamente.	Debe coincidir con el <code>EXPOSE 9000</code> del <code>Dockerfile</code> .
<code>spring.datasource.url = jdbc:postgresql://serve-postgres:5432/db_security</code>	URL de la DB: Define la dirección de la base de datos.	¡El Enganche Principal! <code>serve-postgres</code> debe ser el nombre del servicio de tu contenedor PostgreSQL, y debe estar en la misma red (<code>network_local_server</code>).
<code>spring.datasource.username = postgres</code>	Usuario de la DB: El usuario de la base de datos.	Debe coincidir con el <code>POSTGRES_USER</code> de tus archivos <code>.env</code> de PostgreSQL.
<code>spring.datasource.password = abcd1234</code>	Contraseña de la DB: La contraseña.	¡Alerta de Seguridad! En este ejemplo, esta contraseña está quemada (<code>abcd1234</code>). En un ambiente real, debería ser reemplazada por una variable de entorno inyectada por Docker Compose.
<code>jwt.secret = roITo2yN5V8W...</code>	Secreto JWT: La clave privada usada para	Fundamental para la seguridad del <i>backend</i> . Este secreto debería

	firmar <i>tokens</i> de seguridad.	ser diferente en cada ambiente (dev, prod).
--	------------------------------------	---

Jenkinsfile

El Jenkinsfile representa la automatización del proceso de Integración Continua (CI) y Despliegue Continuo (CD).

Define las etapas que Jenkins debe ejecutar para construir, probar y desplegar el microservicio de manera automática.

Primero genera una versión o etiqueta para cada compilación, luego realiza el despliegue mediante Docker Compose.

De esta forma, se garantiza que cada cambio en el código pase por un flujo controlado y reproducible, facilitando la detección de errores y la trazabilidad de versiones.

Groovy

```

pipeline {
    agent any
    environment {
        DATE_NOW=''
        DOCKER_TAG = "(${DATE_NOW}.${BUILD_NUMBER} | head -c 8)"
        REPOSITORY_URI="backend-security"
        REPOSITORY_URI_APP=""
    }
    stages {
        stage('Construyendo tag') {
            steps {
                script {
                    def now = new Date()
                    DATE_NOW=now.format("yyMMdd.HHmm",
TimeZone.getTimeZone('UTC'))
                    DOCKER_TAG="${DATE_NOW}.${BUILD_NUMBER}"
                    REPOSITORY_URI_APP="$REPOSITORY_URI:$DOCKER_TAG"
                }
            }
        }
        stage('Desplegando') {
            steps {
                sh "sudo CONTAINER_IMAGE=$REPOSITORY_URI_APP docker
compose -f ./devops/docker-compose.yml up -d --build && sudo docker image
prune -f"
            }
        }
    }
}

```

Configuración Inicial (pipeline, agent, environment)

Esta sección define el marco general de ejecución del proceso CI/CD.
 Dentro del bloque `pipeline { ... }` se organiza todo el flujo de integración y despliegue, indicando qué se hace, dónde se ejecuta y qué variables globales se utilizarán.

Línea o bloque	Explicación	Propósito principal
<code>pipeline { ... }</code>	Es la estructura base que contiene todas las etapas (stages) del proceso.	Define el inicio y el fin del flujo CI/CD.
<code>agent any</code>	Permite que Jenkins use cualquier agente (máquina o contenedor disponible) para ejecutar el proceso.	Indica dónde se ejecutará el pipeline.
<code>environment { ... }</code>	Contiene variables globales reutilizables durante todo el proceso (como rutas, nombres o credenciales).	Centraliza configuraciones clave del despliegue.
<code>REPOSITORY_URI="backend-security"</code>	Nombre base del microservicio o imagen Docker a construir.	Identifica el proyecto o módulo.
<code>DATE_NOW</code> y <code>DOCKER_TAG</code>	Variables que se completan en la siguiente etapa con la fecha, hora y número de compilación.	Permiten crear versiones únicas y rastreables.
<code>REPOSITORY_URI_APP</code>	Nombre final de la imagen Docker (ejemplo: <code>backend-security:251104.1841.123</code>).	Determina la etiqueta exacta de la imagen que se construirá y desplegará.

Etapas 1: Construyendo el Tag (Versionamiento)

Esta fase se encarga de crear una etiqueta única para la versión del código que se va a compilar y desplegar.
 Su función es garantizar la trazabilidad y el control de versiones dentro del ciclo de vida del software.

Etapas 2: Desplegando (CI/CD Final)

Línea o bloque	Explicación	Propósito principal
<code>stage('Construyendo tag')</code>	Define la primera etapa del pipeline.	Ordena el flujo del proceso.
<code>def now = new Date()</code>	Obtiene la fecha y hora exacta en la que se ejecuta el build.	Marca el momento de compilación.

<code>DATE_NOW = now.format("yyMMdd.HH:mm", ...)</code>	Convierte la fecha y hora en un formato corto y legible (por ejemplo: 251104.1910).	Facilita la lectura y organización de versiones.
<code>DOCKER_TAG = "\$DATE_NOW.\$BUILD_NUMBER"</code>	Crea una etiqueta combinando fecha, hora y número de compilación (por ejemplo: 251104.1910.123).	Garantiza que cada versión sea irrepetible y rastreable.
<code>REPOSITORY_URI_APP = "\$REPOSITORY_URI:\$DOCKER_TAG"</code>	Une todo para formar el nombre final de la imagen Docker.	Identifica exactamente qué versión del código se está construyendo.

Etapas 2: Despliegue (Ejecución del CI/CD)

En esta etapa se ejecutan los comandos que construyen y despliegan la aplicación de manera automatizada.

El sistema utiliza la versión generada en la etapa anterior y la pone en funcionamiento dentro del entorno configurado.

Línea o bloque	Explicación	Propósito principal
<code>stage('Desplegando')</code>	Define la segunda y última etapa del pipeline.	Marca el inicio del despliegue automático.
<code>sh "..."</code>	Indica que se ejecutará un comando de shell (línea de comandos del sistema operativo).	Permite ejecutar instrucciones reales del servidor.
<code>CONTAINER_IMAGE=\$REPOSITORY_URI_APP</code>	Asigna la imagen construida (con su versión) al entorno de despliegue.	Asegura que el contenedor correcto sea desplegado.
<code>docker compose -f ./devops/docker-compose.yml up -d --build</code>	Orden clave que: lee el archivo <code>docker-compose.yml</code> , construye la imagen, y levanta el contenedor en segundo plano.	Automatiza la construcción y ejecución del servicio.
<code>sudo docker image prune -f</code>	Elimina imágenes obsoletas que ya no se usan.	Mantiene el servidor limpio y optimiza

		espacio en disco.
--	--	-------------------

Conclusiones

El proyecto ENV-DB-ARCHITECTURE representa una solución práctica y ordenada para la gestión de bases de datos PostgreSQL en entornos controlados. Su estructura modular, junto con el uso de Docker, PowerShell y CI/CD con GitHub Actions, permite estandarizar la configuración, despliegue y mantenimiento de bases de datos, garantizando coherencia, seguridad y automatización en cada ambiente.

La aplicación de principios como Infraestructura como Código, Aislamiento de Entornos y Auditoría de Cambios demuestra una arquitectura sólida y moderna, enfocada en la eficiencia y la trazabilidad. Gracias a estas prácticas, el proyecto facilita la colaboración entre equipos, reduce errores humanos y asegura que cada actualización se implemente de forma rápida, confiable y reproducible.